

EECE 7205: Fundamentals of Computer Engineering

Assignment #01

Katherine Tse

Email: tse.kat@northeastern.edu

NUID: 003936627

Due: September 24, 2026

1 Problem 1

Goal: Produce a list of non-overlapping intervals generated from the given list of overlapping intervals.

Inputs: An array of intervals that consist of a start and end value

Outputs: An array of non-overlapping intervals in order of occurrence

Constraints:

- Intervals must be listed in order of start value
- Intervals must not overlap (adjacent intervals are considered overlapping, i.e. $[1,2]$ and $[2,4]$)
- Overlapping intervals should be merged
- No intervals should be dropped or unaccounted for

Assumptions:

- All start and end values are in the same format and are comparable
- Given inputs are not NULL or invalid
- All given values are real time interval (ex. time is not negative)
- All given inputs are in the form (start, end) where start and end are integers

Edge Cases:

- Two intervals are identical (exact same start and end values)
- The input array is empty

Algorithm Logic

1. Sort given list by start time
2. Compare neighboring intervals to see if they overlap
3. If intervals overlap, merge them and then compare to next neighbors to see if there is still overlap that has not been merged

This algorithm makes merging easier as sorting first removes the need to check intervals against all other intervals for overlap.

Algorithm 1.1 Intervals

Input: A array of intervals, *Intervals*

Output: A vector of intervals sorted by start and split to ensure non-overlapping intervals

```
1:  $n \leftarrow \text{SIZEOF}(\text{Intervals})$ 
2: if  $n = 0$  then
3:   return empty list
4: end if
5:  $\text{Sort}(\text{Intervals})$ 
6:  $\text{Sorted} \leftarrow \text{list}$ 
7:  $\text{sorted.APPEND}(\text{Intervals}[0])$ 
8:  $\text{prevInterval} \leftarrow \text{sorted}[\text{SIZEOF}(\text{sorted}) - 1]$ 
9: for  $k \leftarrow 1$  to  $n - 1$  do
10:  if  $\text{prevInterval.end} \geq \text{Intervals}[k].\text{start}$  then
11:    if  $\text{prevInterval.end} < \text{Intervals}[k].\text{end}$  then
12:       $\text{prevInterval.end} \leftarrow \text{Intervals}[k].\text{end}$ 
13:    end if
14:  else if NOT  $(\text{prevInterval.start} = \text{Intervals}[k].\text{start} \text{ AND } \text{prevInterval.end} = \text{Intervals}[k].\text{end})$  then
15:     $\text{sorted.APPEND}(\text{Intervals}[k])$ 
16:     $\text{prevInterval} \leftarrow \text{sorted}[\text{SIZEOF}(\text{sorted}) - 1]$ 
17:  end if
18: end for
19: return  $\text{Sorted}$ 
```

Correctness

The algorithm starts by checking the size of the given array. If the given array of intervals is empty, the code returns an empty list. It then sorts all the given intervals by start value. It then goes through the list and checks if the end of the previous interval is past the start time of the next interval (the value is larger). If it is, the two intervals are merged by updating the end time to the later of the two end times. If there are duplicate intervals, the code will skip one and not add it to the final array.

Complexity Analysis

Time Complexity: $O(n)$

In order to check for overlapping intervals, you need to run through the list once, but you can process the merging within that loop and therefore only need one. This leads to an $O(n)$ runtime.

Space Complexity: Total Space - $O(n)$, Auxiliary Space - $O(1)$

2 Problem 2

Goal: Sort and group a list of items by category (0, 1, 2)

Inputs: An array of values within the determined categories (0, 1, 2)

Outputs: The given array of values sorted and grouped by category

Constraints:

- Array must be sorted in place
- First solution must be count-based
- Second solution must use the one-pass Dutch National Flag algorithm and must not use `std::sort()`

Assumptions:

- The only values within the array are 0, 1, and 2
- Given inputs are not NULL or invalid

Edge Cases:

- The input array is empty
- A given value is not 0, 1, or 2

Algorithm Logic (Counting-based)

1. Use an array to store the count of each category

An array's indexes correspond to the categories

2. Use the counts to determine the last index of each category section

Streamlines rearranging and allows for it to be in-place

3. Rearrange values from the back to front of the array

Best utilizes sections determined above

Algorithm 2.1 Categories: Counting-based

Input: An array of items with category values [containing only (0, 1, 2)], *Items*

Output: *Items* sorted and grouped by category value

```
1:  $n \leftarrow \text{SIZEOF}(\text{Items})$ 
2: if  $n = 0$  then
3:   return
4: end if
5:  $\text{Temp} \leftarrow \text{Items}$ 
6:  $\text{Count} \leftarrow \text{ARRAY}[0, 0, 0]$ 
7: for  $i \leftarrow 0$  to  $n - 1$  do
8:   if  $\text{Items}[i] = 0$  then
9:      $\text{Count}[0] \leftarrow \text{Count}[0] + 1$ 
10:  else if  $\text{Items}[i] = 1$  then
11:     $\text{Count}[1] \leftarrow \text{Count}[1] + 1$ 
12:  else if  $\text{Items}[i] = 2$  then
13:     $\text{Count}[2] \leftarrow \text{Count}[2] + 1$ 
14:  else
15:    return
16:  end if
17: end for
18: for  $j \leftarrow 1$  to  $2$  do
19:    $\text{Count}[j] \leftarrow \text{Count}[j] + \text{Count}[j - 1]$ 
20: end for
21: for  $k \leftarrow n - 1$  to  $0$  do
22:   if  $\text{Temp}[k] = 0$  then
23:      $\text{Items}[\text{Count}[0] - 1] \leftarrow \text{Temp}[k]$ 
24:      $\text{Count}[0] \leftarrow \text{Count}[0] - 1$ 
25:   else if  $\text{Temp}[k] = 1$  then
26:      $\text{Items}[\text{Count}[1] - 1] \leftarrow \text{Temp}[k]$ 
27:      $\text{Count}[1] \leftarrow \text{Count}[1] - 1$ 
28:   else
29:      $\text{Items}[\text{Count}[2] - 1] \leftarrow \text{Temp}[k]$ 
30:      $\text{Count}[2] \leftarrow \text{Count}[2] - 1$ 
31:   end if
32: end for
33: return
```

Algorithm 2.2 Categories: One-pass Dutch National Flag

Input: An array of items with category values [containing only (0, 1, 2)], *Items*

Output: *Items* sorted and grouped by category value

```
1:  $n \leftarrow \text{SIZEOF}(\text{Items})$ 
2: if  $n = 0$  then
3:   return
4: end if
5:  $low \leftarrow 0$ 
6:  $mid \leftarrow 0$ 
7:  $high \leftarrow n - 1$ 
8: while  $mid \leq high$  do
9:   if  $\text{Items}[mid] = 0$  then
10:     $\text{SWAP}(\text{Items}[low], \text{Items}[mid])$ 
11:     $low \leftarrow low + 1$ 
12:     $mid \leftarrow mid + 1$ 
13:   else if  $\text{Items}[mid] = 1$  then
14:     $mid \leftarrow mid + 1$ 
15:   else if  $\text{Items}[mid] = 2$  then
16:     $\text{SWAP}(\text{Items}[mid], \text{Items}[high])$ 
17:     $high \leftarrow high - 1$ 
18:   else
19:    return
20:   end if
21: end while
22: return
```

Correctness (Counting-based)

The algorithm starts by counting the occurrences of each category and storing it in an array. Since the categories are 0, 1, 2, an array creates automatic key value pairs. Next the algorithm calculates the last index of each subcategory which allows for the array to be sorted from back to front and in place. If the given array of items is empty, an empty array will be given back. If a value outside of 0,1, and 2 is given, the code will return an empty array.

Complexity Analysis

Time Complexity (Counting-based): $O(n)$

Throughout this entire algorithm, you are only ever required to traverse the list straight through. There is no need for any additionally nested for loops which allows for an $O(n)$ runtime.

Space Complexity (Counting-based): Total Space - $O(n)$, Auxiliary Space - $O(n)$

Time Complexity (Dutch National Flag): $O(n)$

The while loop in the algorithm runs until the list is sorted. If the list is already sorted, it will do the check until it hits the end of category two (all of values 0 and 1). If the list is reversed (worst case), it will run through the entire list. This leads to a runtime of $O(n)$.

Space Complexity (Dutch National Flag): Total Space - $O(n)$, Auxiliary Space - $O(1)$

3 Problem 3

Goal: Sort a list of integers in order to produce the largest possible number when integers are converted into one long string

Inputs: An vector of nonnegative integers

Outputs: A string that is a concatenation of the inputted array

Constraints:

- Integers cannot be broken up into individual single digits

Assumptions:

- All values in the vector are nonnegative integers
- Given inputs are not NULL or invalid

Edge Cases:

- The vector is empty

Algorithm Logic

Utilizing the insertion sort method to traverse the list and move elements to the correct position allows neighbor comparisons to help achieve largest integer value when combined.

Algorithm 3.1 Integer Concatenation

Input: An vector of nonnegative integers, *Ints*

Output: A string formed from *Ints*

```
1:  $n \leftarrow \text{SIZEOF}(\text{Items})$ 
2: if  $n = 0$  then
3:   return ""
4: end if
5: for  $i \leftarrow 1$  to  $n - 1$  do
6:    $j \leftarrow i - 1$ 
7:    $\text{isLarger} \leftarrow \text{true}$ 
8:   while  $j \geq 0$  AND  $\text{isLarger} = \text{true}$  do
9:      $\text{first} \leftarrow \text{TO\_STRING}(\text{Ints}[j])$ 
10:     $\text{second} \leftarrow \text{TO\_STRING}(\text{Ints}[j + 1])$ 
11:     $\text{current} \leftarrow \text{CONCATENATE}(\text{first}, \text{second})$ 
12:     $\text{swapped} \leftarrow \text{CONCATENATE}(\text{second}, \text{first})$ 
13:    if  $\text{TO\_INT}(\text{swapped}) > \text{TO\_INT}(\text{current})$  then
14:       $\text{SWAP}(\text{Ints}[j + 1], \text{Ints}[j])$ 
15:       $j \leftarrow j - 1$ 
16:    else
17:       $\text{isLarger} \leftarrow \text{false}$ 
18:    end if
19:  end while
20: end for
21:  $\text{final} \leftarrow ""$ 
22: for  $k \leftarrow 0$  to  $n - 1$  do
23:    $\text{final} \leftarrow \text{final} + \text{TO\_STRING}(\text{Ints}[k])$ 
24: end for
25: return
```

Correctness

Utilizing insertion sort, the algorithm compares the order of two integers to evaluate which concatenation creates the largest integer and sorts the list accordingly. At the end, it casts all of the integers to strings and concatenates them together. If the given list of integers is empty, an empty string is returned.

Complexity Analysis

Time Complexity: $O(n^2)$

The outer for loop runs through the entire list. The inner while loop checks the current value against it's neighbors to ensure that the order of the values produces the optimal result. Since being able to access the exact neighbors is important, a runtime of $O(n^2)$ is required.

Space Complexity: Total Space - $O(n)$, Auxiliary Space - $O(1)$

4 Problem 4

Goal: Sort a list of strings by grouping all of the anagrams together.

Inputs: A vector of strings

Outputs: A collection of strings where all anagrams are group together

Constraints:

- Words within a grouping should maintain the order they were originally given in

Assumptions:

- Given inputs are not NULL or invalid
- Given inputs are all valid strings

Edge Cases:

- The list is empty

Algorithm Logic

1. Utilize a hashmap data strcuture with arrays as the values

This allows you to store key value pairs

2. Traverse the list, sort each word into each letter alphabetically, and add the entry to the hash table

By sorting the words alphabetically, anagrams can be easily compared and keyed in the hashtable

3. Combine the values from the hashmap to create the final sorted list

Algorithm 4.1 Anagrams

Input: A list of strings, *Words*

Output: *Words* grouped by anagrams

```
1:  $n \leftarrow \text{SIZEOF}(\text{Words})$ 
2: if  $n = 0$  then
3:   return Words
4: end if
5:  $\text{keys} \leftarrow \text{empty list}$ 
6:  $\text{map} \leftarrow \text{CREATE\_HASHMAP}()$ 
7: for  $i \leftarrow 0$  to  $n - 1$  do
8:    $\text{current} \leftarrow \text{Words}[i]$ 
9:    $\text{Chars} \leftarrow \text{TO\_CHAR\_ARRAY}(\text{current})$ 
10:   $\text{sort}(\text{Chars})$ 
11:   $m \leftarrow 0$ 
12:   $\text{wasAdded} \leftarrow \text{false}$ 
13:  while  $m \leq \text{SIZEOF}(\text{keys}) - 1$  AND  $\text{wasAdded} = \text{false}$  do
14:    if  $\text{SIZEOF}(\text{Chars}) = \text{SIZEOF}(\text{keys}[m])$  AND  $\text{Chars} = \text{keys}[m]$  then
15:       $\text{map.GET}(\text{keys}[m]).\text{APPEND}(\text{current})$ 
16:       $\text{wasAdded} \leftarrow \text{true}$ 
17:    end if
18:     $m \leftarrow m + 1$ 
19:  end while
20:  if  $\text{wasAdded} = \text{false}$  then
21:     $\text{map.SET}(\text{Chars}, \text{VECTOR}[\text{current}])$ 
22:     $\text{keys.APPEND}(\text{Chars})$ 
23:  end if
24: end for
25:  $\text{grouped} \leftarrow \text{VECTOR}[\ ]$ 
26: for  $z \leftarrow 0$  to  $\text{SIZEOF}(\text{keys}) - 1$  do
27:    $\text{grouped} \leftarrow \text{CONCATENATE}(\text{grouped}, \text{map.GET}(\text{keys}[z]))$ 
28: end for
29: return grouped
```

Correctness

The algorithm runs through the entire list. It starts by sorting the letters in the current item alphabetically. These sorted words become the keys for the hash map. After sorting the letters, the algorithm compares the key to the existing hash map keys. If a match is found, the original word is added to the array value for that key is updated. If a match is not found, the new key and word are added to the hash map. The algorithm ends by concatenating all of the arrays from the hash map back into a single array. If an empty array is given as input, an empty array is returned.

Complexity Analysis

Time Complexity: Best - $O(n)$, Average/Worst - $O(n^2)$

The algorithm traverses through the entire list at least once. The second loop is dependent on how many unique anagrams are within the list. In the best case scenario, all of the words are anagrams and the check within the while loop only ever runs once per list item. In the worst case, all of the items are unique and the while loop runs through the entire list in the end. This leads to a worst case and average case of $O(n^2)$ runtime.

Space Complexity: Total Space - $O(n)$, Auxiliary Space - $O(n)$

5 Problem 5

Goal: Determine if any meeting intervals overlap

Inputs: Intervals that consist of a start and end value

Outputs: A boolean indicating whether or not a person can attend all the meetings given [true -> yes, false -> no]

Constraints:

- Meetings that touch are considered able to attend (start time matches an end time)

Assumptions:

- All start and end values are in the same format and are comparable
- Intervals are given in a list/array format
- All intervals are valid times (i.e. no negative time)
- Given inputs are not NULL or invalid
- All given inputs are in the form (start, end) where start and end are integers

Edge Cases:

- Two intervals are identical (exact same start and end values)
- The input array is empty

Algorithm Logic

1. Sort given list by start dates using merge sort

Ensures that overlapping intervals are adjacent to each other.

2. Check if there are overlapping intervals

Algorithm 5.1 Availability

Input: A array of intervals, *Intervals*

Output: A boolean indicating if a person can attend all meetings in *Intervals* [true -> yes, false -> no]

```
1:  $n \leftarrow \text{SIZEOF}(\text{Intervals})$ 
2: if  $n = 0$  then
3:   return true
4: end if
5: Sort(Intervals, 0,  $n - 1$ )
6: for  $k \leftarrow 0$  to  $n - 2$  do
7:   if Intervals[ $k$ ].end > Intervals[ $k + 1$ ].start then
8:     return false
9:   else if Intervals[ $k$ ].start = Intervals[ $k + 1$ ].start AND Intervals[ $k$ ].end = Intervals[ $k + 1$ ].end then
10:    return false
11:   end if
12: end for
13: return true
```

Correctness

The algorithm begins by sorting all of the given intervals by start time. This aligns all meetings with the same or close start times to be near each other. The algorithm then runs through the list until it reaches the end or find an overlapping meeting. If an overlapping meeting exists, the algorithm returns a boolean representing this information. If the given array is empty, the program returns true stating that a person can attend all meetings scheduled.

Complexity Analysis

Time Complexity: Best - $O(1)$, Average/Worst - $O(n)$

Since the goal of the algorithm is just to determine whether or not an overlap exists, the best case scenario involves the first two intervals overlapping which would result in a $O(1)$ runtime. However, any other case would result in some level of iterating through the list which makes the average and worst case runtime $O(n)$.

Space Complexity: Total Space - $O(n)$, Auxiliary Space - $O(1)$

6 Problem 6

Goal: Find the requested number of points (x, y) closets to the origin

Inputs:

- Points (x,y)
- The number of desired points to return

Outputs: A list of the given length of points closet to the origin

Constraints:

- Use sorting solution for first algorithm
- Use heap-based solution for second algorithm

Assumptions:

- The origin is at point (0,0)
- All given points are valid integer coordinates in the format (x, y)
- All given inputs are not NULL or invalid
- Points are given in a list/array format
- All given points are real (x,y) integer coordinates
- K is a valid positive integer

Edge Cases:

- List of points is smaller than given desired number
- List of points is empty

Algorithm Logic (Sorting)

1. Sort the list of points using merge sort
2. Grab only the number of points requested

This sorting algorithm allows for the fastest sort to streamline the overall algorithm

Algorithm Logic (heap-based)

1. Initialize a min heap with the given points
2. Pop off the requested number of points off of the heap

The heap data structure is sorted as items are added and utilizing a min heap makes finding the desired values quick and easy.

Algorithm 6.1 Origin: Sort

Input: List of points (x, y), *Points*; desired number of points returned, *k*

Output: Subset of *Points* made up of the *k* points closet to the origin (0,0)

```
1: Function merge(vals : ARRAY[ ], left : integer, mid : integer, right : integer)
    n1  $\leftarrow$  mid - left + 1
    n2  $\leftarrow$  right - mid
    ltemp  $\leftarrow$  VECTOR[ ]
    rtemp  $\leftarrow$  VECTOR[ ]
    for l  $\leftarrow$  0 to n1 - 1 do
        ltemp.APPEND(vals[left + l])
    end for
    for r  $\leftarrow$  0 to n2 - 1 do
        rtemp.APPEND(vals[mid + 1 + r])
    end for
    i  $\leftarrow$  0
    j  $\leftarrow$  0
    k  $\leftarrow$  left
    while i < n1 AND j < n2 do
        ldist  $\leftarrow$  ltemp[i].x2 + ltemp[i].y2
        rdist  $\leftarrow$  rtemp[j].x2 + rtemp[j].y2
        if ldist  $\leq$  rdist then
            vals[k]  $\leftarrow$  ltemp[i]
            i  $\leftarrow$  i + 1
        else
            vals[k]  $\leftarrow$  rtemp[j]
            j  $\leftarrow$  j + 1
        end if
        k  $\leftarrow$  k + 1
    end while
    while i < n1 do
        vals[k]  $\leftarrow$  ltemp[i]
        i  $\leftarrow$  i + 1
        k  $\leftarrow$  k + 1
    end while
    while j < n2 do
        vals[k]  $\leftarrow$  rtemp[j]
        j  $\leftarrow$  j + 1
        k  $\leftarrow$  k + 1
    end while
2: End Function
```

Algorithm 6.1 Origin: Sort (continued)

```
3: Function mergeSort(vals : ARRAY [ ], left : integer, right : integer)  
    if left  $\geq$  right then  
        return  
    end if  
    mid  $\leftarrow$  left + (right - left)/2  
    mergeSort(vals, left, mid)  
    mergeSort(vals, mid + 1, right)  
    merge(vals, left, mid, right)  
4: End Function  
5: if k == 0 then  
6:     return [ ]  
7: end if  
8: n  $\leftarrow$  SIZEOF(Points)  
9: if n  $\leq$  k then  
10:    return Points  
11: end if  
12: mergeSort(Points, 0, n - 1)  
13: return Points[0, k-1]
```

Algorithm 6.2 Origin: Heap

Input: List of points (x, y), *Points*; desired number of points returned, *k*

Output: Subset of *Points* made up of the *k* points closet to the origin (0,0)

```
1: if  $k == 0$  then
2:   return [ ]
3: end if
4:  $n \leftarrow \text{SIZEOF}(\text{Points})$ 
5: if  $n \leq k$  then
6:   return Points
7: end if
8:  $\text{heap} \leftarrow \text{VECTOR}[\text{Points}[0]]$ 
9: for  $i \leftarrow 1$  to  $n - 1$  do
10:   $\text{heap}.\text{APPEND}(\text{Points}[i])$ 
11:   $\text{index} \leftarrow \text{SIZEOF}(\text{heap}) - 1$ 
12:   $\text{current} \leftarrow \text{heap}[\text{index}].x^2 + \text{heap}[\text{index}].y^2$ 
13:   $\text{parent} \leftarrow \text{heap}[(\text{index} - 1)/2].x^2 + \text{heap}[(\text{index} - 1)/2].y^2$ 
14:  while  $\text{index} > 0$  AND  $\text{current} < \text{parent}$  do
15:     $\text{SWAP}(\text{heap}[\text{index}], \text{heap}[(\text{index} - 1)/2])$ 
16:     $\text{index} \leftarrow (\text{index} - 1)/2$ 
17:     $\text{parent} \leftarrow \text{heap}[(\text{index} - 1)/2].x^2 + \text{heap}[(\text{index} - 1)/2].y^2$ 
18:  end while
19: end for
20:  $h \leftarrow \text{SIZEOF}(\text{heap})$ 
21:  $\text{final} \leftarrow \text{VECTOR}[\ ]$ 
22: while  $\text{SIZEOF}(\text{final}) < k$  do
23:   $\text{final}.\text{APPEND}[\text{heap}[0]]$ 
24:   $\text{SWAP}(\text{heap}[0], \text{heap}[h - 1])$ 
25:   $h \leftarrow h - 1$ 
26:   $\text{ind} \leftarrow 0$ 
27:   $\text{fixed} \leftarrow \text{false}$ 
28:  while  $\text{fixed} = \text{false}$  do
29:     $\text{left} \leftarrow 2 * \text{ind} + 1$ 
30:     $\text{right} \leftarrow 2 * \text{ind} + 2$ 
31:     $\text{min} \leftarrow \text{ind}$ 
32:     $\text{minDist} \leftarrow \text{heap}[\text{min}].x^2 + \text{heap}[\text{min}].y^2$ 
33:    if  $\text{left} < h$  AND  $\text{heap}[\text{left}] < \text{heap}[\text{min}]$  then
34:       $\text{min} \leftarrow \text{left}$ 
35:    end if
36:    if  $\text{right} < h$  AND  $\text{heap}[\text{right}] < \text{heap}[\text{min}]$  then
37:       $\text{min} \leftarrow \text{right}$ 
38:    end if
```

Algorithm 6.2 Origin: Sort (continued)

```
39:      if  $min \neq ind$  then
40:           $SWAP(heap[min], heap[ind])$ 
41:           $ind \leftarrow min$ 
42:      else
43:           $fixed \leftarrow true$ 
44:      end if
45:  end while
46: end while
47: return  $final$ 
```

Correctness (Sorting)

The algorithm uses merge sort to efficiently sort all points by their calculated distance from the origin. From there, the correct subsection of points [0 to k] is returned. If an empty list is given or if the size of the list is smaller than the given desired length, the original array is returned sorted.

Correctness (Heap-based)

The algorithm starts by sorting all of the points by distance from the origin by putting them into a min-heap. It then pops off the desired number of values to return. If an empty list is given or if the list is shorter than the number of items being requested, the entire list is returned.

Complexity Analysis

Time Complexity (Sorting): $O(n \log n)$

By utilizing and altering the merge sort strategy, the algorithm is able to have a runtime of $O(n \log n)$ for best, average, and worst case. To return the requested number of points, there is no additional complexity that changes the overall runtime.

Space Complexity (Sorting): Total Space - $O(n)$, Auxiliary Space - $O(n)$

Time Complexity (Heap-based): $O(n \log n)$

Utilizing a heap allows for an $O(n \log n)$ runtime because the creation and redistribution of an heap is done by splitting data based on each node. Every time you extract a node, you still maintain the $O(n \log n)$ runtime, and because Big-O notation ignores constants, the number of k iterations requested to find that number of points gets ignored.

Space Complexity (Heap-based): Total Space - $O(n)$, Auxiliary Space - $O(n)$

7 Problem 7

Goal: Find the requested number of values with the highest frequencies

Inputs:

- An integer array
- The number of desired values to return

Outputs: A list of values with the highest frequencies of the desired length

Constraints:

- When frequencies tie, the smaller value should be returned first

Assumptions:

- Returned values are listed in order of frequency
- All given inputs are not NULL or invalid
- All given value are integers
- K is a valid positive integer

Edge Cases:

- The number of unique values is less than the requested return amount
- The list is empty

Algorithm Logic

1. Sort the integer array using merge sort

This makes it so you can calculate frequency by counting the values in the array

2. Count the frequency of each number and store in an array as key value pairs (custom structure)

This allows you to sort by frequency while also keeping the original value linked.

3. Sort the frequencies from largest to smallest using merge sort
4. Pull the desired number of values from the list sorted by frequency.

Algorithm 7.1 Frequency

Input: An integer Array, *Nums*; The number of desired high frequency values, *k*
Output: A list of length *k* that includes the values of highest frequency from *Nums*

```
1: Structure CountPair
2:   value : integer
3:   count : integer
4: End Structure
5: n  $\leftarrow$  SIZEOF(Nums)
6: if n = 0 then
7:   return Nums
8: end if
9: Sort(Nums)
10: key  $\leftarrow$  Nums[0]
11: count  $\leftarrow$  1
12: frequencyPairs  $\leftarrow$  VECTOR[ ]
13: for i  $\leftarrow$  1 to n - 1 do
14:   if Nums[i] = key then
15:     count  $\leftarrow$  count + 1
16:   else
17:     frequencyPairs.append(new CountPair(key, count))
18:     key  $\leftarrow$  Nums[i]
19:     Count  $\leftarrow$  1
20:   end if
21: end for
22: frequencyPairs.append(new CountPair(key, count))
23: Sort(frequencyPairs)
24: f  $\leftarrow$  SIZEOF(frequencyPairs)
25: final  $\leftarrow$  VECTOR[ ]
26: for y  $\leftarrow$  0 to k - 1 do
27:   if y  $\geq$  f then
28:     return final
29:   end if
30:   final.APPEND(frequencyPairs[y].value)
31: end for
32: return final
```

Correctness

The algorithm begins by sorting the given integer array using merge sort. It then performs a count on all of the values in the array and stores this count in another array using a custom count value structure. It then sorts the list of count value pairs by count and returns the requested number of integers. If the number of values requested is greater than the number of values in the array, the entire array is returned. If the given array is empty, the empty array is returned.

Complexity Analysis

Time Complexity: $O(n)$

Since we are able to sort the list before performing our calculations. We are able to do a count of all integers with one pass through the list which results in a runtime of $O(n)$. Additionally, accessing the k number of top values takes at worst a single pass through the list so the runtime stays at $O(n)$.

Space Complexity: Total Space - $O(n)$, Auxiliary Space - $O(1)$

8 Problem 8

Goal: Combine a set of sorted arrays into a single fully sorted array

Inputs: Sorted integer arrays

Outputs: A single sorted integer array made up of all inputted arrays

Constraints:

- Algorithm must not repeatedly sort the complete output

Assumptions:

- The inputs are given in a list/array
- All integer arrays are valid/ not NULL and only have integers

Edge Cases:

- No arrays are given
- A given array is empty

Algorithm Logic

1. Combine two arrays by comparing the next element from each array and adding them in sorted order
 2. Continue to add additional arrays to the existing combined array using the same method
- This strategy ensures the arrays maintain their sorted state as new arrays are merged and does not require a secondary sort through the final combined array.

Algorithm 8.1 Combining Arrays

Input: An array of k sorted integer arrays, $intArrs$

Output: A single sorted array of all values from the arrays in $intArrs$

```
1: if SIZEOF( $intArrs$ ) = 0 then
2:   return  $ARRAY[]$ 
3: end if
4: Function  $mergeArrs(arr1 : ARRAY[< integer >], arr2 : ARRAY[< integer >])$ 
    $currVal1 \leftarrow 0$ 
    $currVal2 \leftarrow 0$ 
    $final \leftarrow \text{empty list}$ 
   while  $currVal1 < SIZEOF(arr1)$  AND  $currVal2 < SIZEOF(arr2)$  do
     if  $arr1[currVal1] \leq arr2[currVal2]$  then
        $final.APPEND(arr1[currVal1])$ 
        $currVal1 \leftarrow currVal1 + 1$ 
     else
        $final.APPEND(arr2[currVal2])$ 
        $currVal2 \leftarrow currVal2 + 1$ 
     end if
   end while
   if  $currVal1 < SIZEOF(arr1)$  then
      $final.APPEND(arr1[currVal1 :])$ 
   end if
   if  $currVal2 < SIZEOF(arr2)$  then
      $final.APPEND(arr2[currVal2 :])$ 
   end if
   return  $final$ 
5: End Function
6:  $allArrs \leftarrow intArrs[0]$ 
7: for  $k \leftarrow 1$  to  $SIZESOF(intArrs) - 1$  do
8:    $allArrs \leftarrow mergeArrs(allArrs, intArrs[k])$ 
9: end for
10: return  $allArrs$ 
```

Correctness

This algorithm combines two arrays, sorting them as they are combined. It runs through all of the given arrays by merging the first two and then merging each subsequent one into the outcome of the previous merge. This creates an already sorted array that does not need to be resorted at the end. If no arrays are given, an empty array is returned. If one of the arrays is empty, the merge will result in no changes to the other existing array.

Complexity Analysis

Time Complexity: $O(n)$

Since the goal of the algorithm is to merge a list of sorted arrays, you only need to traverse the list of arrays once to access said arrays. This results in a runtime of $O(n)$.

Space Complexity: Total Space - $O(n)$, Auxiliary Space - $O(1)$

9 Problem 9

Goal: Return the kth-largest element from an unsorted array

Inputs:

- An unsorted integer array
- An integer k

Outputs: The kth-largest element from the array

Constraints:

- The algorithm must not sort the entire array

Assumptions:

- All values in the array are valid integers
- k is a positive integer

Edge Cases:

- The array is empty
- k is larger than the array size

Algorithm Logic

1. Begin bringing the largest values to the front of the array
2. Continue to sort until the index of the value matches the requested number

This method does not presort the entire list and instead only sorts until the desired value is found.

Algorithm 9.1 kth-largest

Input: An array of integers, *ints*; an integer, *k*

Output: An integer that is the *k*th-largest element in *ints*

```
1: if SIZEOF(ints) = 0 then
2:   return 0
3: end if
4: final  $\leftarrow$  0
5: if k < SIZEOF(ints) then
6:   for i  $\leftarrow$  0 to k - 1 do
7:     maxIDX  $\leftarrow$  i
8:     for j  $\leftarrow$  i + 1 to SIZEOF(ints) - 1 do
9:       if ints[j] > ints[maxIDX] then
10:        maxIDX  $\leftarrow$  j
11:      end if
12:    end for
13:    SWAP(ints[i], ints[maxIDX])
14:    final  $\leftarrow$  ints[i]
15:  end for
16: else
17:   minIDX  $\leftarrow$  0
18:   for z  $\leftarrow$  1 to SIZEOF(ints) - 1 do
19:     if ints[z] < ints[minIDX] then
20:       minIDX  $\leftarrow$  z
21:     end if
22:   end for
23:   final  $\leftarrow$  ints[minIDX]
24: end if
25: return final
```

Correctness

The algorithm begins by checking if the requested index is smaller than or equal to the length of the array. If it is, it searches the array for the largest array and then disregards it until it reaches the desired requested value. If the value is larger than the size of the array, the minimum value of the array is returned. If the array is empty, 0 is returned.

Complexity Analysis

Time Complexity: Best - $O(n)$, Average/Worst - $O(n^2)$

The goal of the algorithm is to find the *k*th largest value without sorting the entire list to start. This allows for a best time of $O(n)$ assuming that the value requested is the largest or smallest value in the array. However, the average and worse case end up evaluating to $O(n^2)$ because in order to find the maximum values without sorting the entire list you need to determine each one as you go.

Space Complexity: Total Space - $O(n)$, Auxiliary Space - $O(1)$

10 Problem 10

Goal: Count the number of pairs in the array that are out of order ($i < j$ and $\text{value}[i] > \text{value}[j]$)

Inputs: An array of values

Outputs: An integer representing the number of out order pairs

Constraints:

- Algorithm must be based on Merge Sort algorithm
- Output should be a 64-bit integer

Assumptions:

- All values in the given array are comparable
- All values in the given array are valid/not NULL

Edge Cases:

- The array is empty

Algorithm Logic

The problem requested a modified merge sort algorithm. A count is added when an inversion swap is made.

Algorithm 10.1 Inversions

Input: An array of values, *vals*

Output: A 64-bit integer representing the inversion count of the given array *vals*

1: **Function** *merge*(*vals* : *ARRAY*[], *left* : *integer*, *mid* : *integer*, *right* : *integer*)

$n1 \leftarrow mid - left + 1$

$n2 \leftarrow right - mid$

$ltemp \leftarrow VECTOR[]$

$rtemp \leftarrow VECTOR[]$

for $l \leftarrow 0$ to $n1 - 1$ **do**

$ltemp.APPEND(vals[left + l])$

end for

for $r \leftarrow 0$ to $n2 - 1$ **do**

$rtemp.APPEND(vals[mid + 1 + r])$

end for

$i \leftarrow 0$

$j \leftarrow 0$

$k \leftarrow left$

$invCount \leftarrow 0$

while $i < n1$ AND $j < n2$ **do**

if $ltemp[i] \leq rtemp[j]$ **then**

$vals[k] \leftarrow ltemp[i]$

$i \leftarrow i + 1$

else

$vals[k] \leftarrow rtemp[j]$

$j \leftarrow j + 1$

$invCount \leftarrow invCount + (n1 - i)$

end if

$k \leftarrow k + 1$

end while

while $i < n1$ **do**

$vals[k] \leftarrow ltemp[i]$

$i \leftarrow i + 1$

$k \leftarrow k + 1$

end while

while $j < n2$ **do**

$vals[k] \leftarrow rtemp[j]$

$j \leftarrow j + 1$

$k \leftarrow k + 1$

end while

return $invCount$

2: **End Function**

Algorithm 10.1 Inversions (continued)

```
3: Function mergeCount(vals : ARRAY[ ], left : integer, right : integer)
    invCount  $\leftarrow$  0
    if left  $\geq$  right then
        return
    end if
    mid  $\leftarrow$  left + (right - left)/2
    invCount  $\leftarrow$  invCount + mergeCount(vals, left, mid)
    invCount  $\leftarrow$  invCount + mergeCount(vals, mid + 1, right)
    invCount  $\leftarrow$  invCount + merge(vals, left, mid, right)
    return invCount
4: End Function
5: n  $\leftarrow$  SIZEOF(vals)
6: if n = 0 then
7:     return 0
8: end if
9: return mergeCount(vals, 0, n-1)
```

Correctness

The algorithm is a modified version of merge sort that take count of every time an item is not already sorted. Every time a value from right is swapped, the counter is increased by one. If the given array is empty, 0 is returned.

Complexity Analysis

Time Complexity: $O(n \log n)$

The algorithm is a modified version of merge sort and therefore has a runtime of $O(n \log n)$

Space Complexity: Total Space - $O(n)$, Auxiliary Space - $O(n)$

11 Test Suite Results

The complete test suite for Assignment 1 was executed, covering all ten problems. Here is the output.

Assignment 1 Test Suite

Testing Problem 1

Input 1: {1, 3}, {2, 6}, {8, 10}, {15, 18}

Execution time: 0ms

[PASS] tests/test_problems.cpp:37 - {{1, 6}, {8, 10}, {15, 18}} ==
{{1, 6}, {8, 10}, {15, 18}}

Input 2: {5, 10}, {1, 2}, {9, 15}, {2, 3}, {15, 20}

Execution time: 0ms

[PASS] tests/test_problems.cpp:50 - {{1, 3}, {5, 20}} == {{1, 3},
{5, 20}}

Input 3: {}

```

Execution time: 0ms
[PASS] tests/test_problems.cpp:63 - {} == {}

Testing Problem 2
-----
Input 1: {2, 0, 2, 1, 1, 0}
Count Execution Time: 0ms
[PASS] tests/test_problems.cpp:83 - {0, 0, 1, 1, 2, 2} == {0, 0, 1,
    1, 2, 2}
One-Pass Dutch National Flag Execution Time: 0ms
[PASS] tests/test_problems.cpp:91 - {0, 0, 1, 1, 2, 2} == {0, 0, 1,
    1, 2, 2}
Input 2: {}
Count Execution Time: 0ms
[PASS] tests/test_problems.cpp:105 - {} == {}
One-Pass Dutch National Flag Execution Time: 0ms
[PASS] tests/test_problems.cpp:113 - {} == {}
Input 3: {1, 1, 1}
Count Execution Time: 0ms
[PASS] tests/test_problems.cpp:127 - {1, 1, 1} == {1, 1, 1}
One-Pass Dutch National Flag Execution Time: 0ms
[PASS] tests/test_problems.cpp:135 - {1, 1, 1} == {1, 1, 1}

Testing Problem 3
-----
Input 1: {3, 30, 34, 5, 9}
Execution Time: 0ms
[PASS] tests/test_problems.cpp:154 - expt1 == 9534330
Input 2: {}
Execution Time: 0ms
[PASS] tests/test_problems.cpp:167 - expt2 ==
Input 3: {0, 0, 0, 0}
Execution Time: 0ms
[PASS] tests/test_problems.cpp:180 - expt3 == 0

Testing Problem 4
-----
Input 1: {"eat", "tea", "tan", "ate", "nat", "bat"}
Execution Time: 0ms
[PASS] tests/test_problems.cpp:199 - {{eat, tea, ate}, {tan, nat}, {
    bat}} == {{eat, tea, ate}, {tan, nat}, {bat}}
Input 2: {"", ""}
Execution Time: 0ms
[PASS] tests/test_problems.cpp:212 - {{, }} == {{, }}
Input 3: {"ab", "ba", "abc"}
Execution Time: 0ms

```

```
[PASS] tests/test_problems.cpp:225 - {{ab, ba}, {abc}} == {{ab, ba},
    {abc}}
```

Testing Problem 5

```
Input 1: {0, 24}, {5, 10}, {15, 20}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:244 - expt1 == 0
```

```
Input 2: {}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:257 - expt2 == 1
```

```
Input 3: {5, 10}, {1, 5}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:270 - expt3 == 1
```

Testing Problem 6

```
Input 1: {1, 3}, {-2, 2}, {5, 8}, {0, 1}
```

```
Sort Execution Time: 0ms
```

```
[PASS] tests/test_problems.cpp:291 - {(0, 1), (-2, 2)} == {(0, 1),
    (-2, 2)}
```

```
Heap Execution Time: 0ms
```

```
[PASS] tests/test_problems.cpp:299 - {(0, 1), (-2, 2)} == {(0, 1),
    (-2, 2)}
```

```
Input 2: {3, 3}, {5, -1}, {-2, 4}
```

```
Sort Execution Time: 0ms
```

```
[PASS] tests/test_problems.cpp:314 - {(3, 3), (5, -1), (-2, 4)} ==
    {(3, 3), (5, -1), (-2, 4)}
```

```
Heap Execution Time: 0ms
```

```
[PASS] tests/test_problems.cpp:322 - {(3, 3), (5, -1), (-2, 4)} ==
    {(3, 3), (5, -1), (-2, 4)}
```

```
Input 3: {1, 1}
```

```
Sort Execution Time: 0ms
```

```
[PASS] tests/test_problems.cpp:337 - {} == {}
```

```
Heap Execution Time: 0ms
```

```
[PASS] tests/test_problems.cpp:345 - {} == {}
```

Testing Problem 7

```
Input 1: {1, 2, 1, 1, 3, 2}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:365 - {1, 2} == {1, 2}
```

```
Input 2: {4, 5, 6, 6, 5, 4, 7}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:379 - {4, 5, 6} == {4, 5, 6}
```

```
Input 3: {}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:393 - {} == {}
```

Testing Problem 8

```
Input 1: {1, 4, 7}, {2, 5, 8, 9}, {3,6}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:412 - {1, 2, 3, 4, 5, 6, 7, 8, 9} ==  
    {1, 2, 3, 4, 5, 6, 7, 8, 9}
```

```
Input 2: {}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:425 - {} == {}
```

```
Input 3: {2, 2, 3}, {2, 2}, {1}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:438 - {1, 2, 2, 2, 2, 3} == {1, 2, 2,  
    2, 2, 3}
```

Testing Problem 9

```
Input 1: {3, 2, 1, 5, 6, 4}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:458 - expt1 == 5
```

```
Input 2: {3, 3, 3, 2, 1, 0}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:472 - expt2 == 3
```

```
Input 3: {3, 7, 1, 11}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:486 - expt3 == 1
```

Testing Problem 10

```
Input 1: {8, 4, 2, 1}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:505 - expt1 == 6
```

```
Input 2: {2, 4, 1, 3, 5}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:518 - expt2 == 3
```

```
Input 3: {}
```

```
Execution time: 0ms
```

```
[PASS] tests/test_problems.cpp:531 - expt3 == 0
```

36/36 checks passed

All 36 checks passed successfully, confirming correct behavior for each problem across the tested edge cases (typical inputs, empty inputs, and inputs with duplicate or negative values).